

OOPs assignment 6

Submitted by: Mehma Kaur Chadha

Roll Number 1024150080

Batch:2013

1. Q1 Implement a class ***Library*** that contains **an array of *Book objects***. The ***Book*** class should have attributes like **title, author, and ISBN**. The ***Library*** class should provide following methods:
 - a. **bool addNewBook(title,author,ISBN)** to add the new book in the book list. The return type should be Boolean to check whether the book is added or not. This function should take the argument using pass-by-reference method. The attribute of the class (title, author, and ISBN) and the function arguments of function **addNewBook** should be same. Use the **scope resolution operator** to access the class data members.
 - b. **bool removeBooks(ISBN)** to remove the book using the input parameter ISBN. The return type should be Boolean to check whether the book is removed or not. This function also should take the argument using pass-by- reference method. Define function outside the class using scope resolution operator.
 - c. **displayDetails()** to list the books.

Write a main function to demonstrate adding, removing, and displaying books from the library. Add at least 5 books to demonstrate the adding book, 1 book to remove from the list.

main.cpp

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Book {
6  public:
7      string title, author, ISBN;
8  };
9
10 class Library {
11 private:
12     Book books[10];
13     int count = 0;
14
15 public:
16     bool addNewBook(const string &title, const string &author, const string
17         if (count >= 10) return false;
18
19         books[count].title = title;
20         books[count].author = author;
21         books[count].ISBN = ISBN;
22         count++;
23         return true;
24     }
25
26     bool removeBooks(const string &ISBN);
27
28     void displayDetails() {
29         for (int i = 0; i < count; i++) {
```

main.cpp

```
26     bool removeBooks(const string &ISBN);
27
28     void displayDetails() {
29         for (int i = 0; i < count; i++) {
30             cout << books[i].title << " | "
31                 << books[i].author << " | "
32                 << books[i].ISBN << endl;
33         }
34     }
35 };
36
37 bool Library::removeBooks(const string &ISBN) {
38     for (int i = 0; i < count; i++) {
39         if (books[i].ISBN == ISBN) {
40             for (int j = i; j < count - 1; j++)
41                 books[j] = books[j + 1];
42             count--;
43             return true;
44         }
45     }
46     return false;
47 }
48
49 int main() {
50     Library lib;
51
52     lib.addNewBook("A", "X", "111");
53     lib.addNewBook("B", "Y", "222");
54     lib.addNewBook("C", "Z", "333");
```

```

48
49 int main() {
50     Library lib;
51
52     lib.addNewBook("A", "X", "111");
53     lib.addNewBook("B", "Y", "222");
54     lib.addNewBook("C", "Z", "333");
55     lib.addNewBook("D", "P", "444");
56     lib.addNewBook("E", "Q", "555");
57
58     lib.displayDetails();
59
60     cout << "\nAfter removal:\n";
61     lib.removeBooks("333");
62
63     lib.displayDetails();
64 }

```

input

```

A | X | 111
B | Y | 222
C | Z | 333
D | P | 444
E | Q | 555

After removal:
A | X | 111
B | Y | 222
D | P | 444
E | Q | 555

...Program finished with exit code 0
Press ENTER to exit console.

```

Q2 Implement the question 1 using constructor. Do the following modification:

- Add default constructor, parameterized constructor, and copy constructor to add the details of the book using the same parameters, i.e., **title, author, and ISBN**.

- b. Use the “this” pointer instead of scope resolution operator to access the data members of the class inside the constructor and function definition.
- c. Create function **bool removeBooks(ISBN)** same as the question 1.
- d. **displayDetails()** same as question 1.
- e. Create array of objects using 1. Initializer list, 2. Dynamic initialization

main.cpp

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Book {
6  public:
7      string title, author, ISBN;
8
9      Book() {}
10
11     Book(string title, string author, string ISBN) {
12         this->title = title;
13         this->author = author;
14         this->ISBN = ISBN;
15     }
16
17     Book(const Book &b) {
18         this->title = b.title;
19         this->author = b.author;
20         this->ISBN = b.ISBN;
21     }
22 };
23
24 class Library {
25 private:
26     Book *books;
27     int count;
28
29 public:
30     Library(int size) {
```

main.cpp

```
23
24 class Library {
25 private:
26     Book *books;
27     int count;
28
29 public:
30     Library(int size) {
31         books = new Book[size];
32         count = size;
33     }
34
35     bool removeBooks(const string &ISBN) {
36         for (int i = 0; i < count; i++) {
37             if (books[i].ISBN == ISBN) {
38                 books[i] = books[count - 1];
39                 count--;
40                 return true;
41             }
42         }
43         return false;
44     }
45
46     void displayDetails() {
47         for (int i = 0; i < count; i++) {
48             cout << books[i].title << " "
49                 << books[i].author << " "
50                 << books[i].ISBN << endl;
51         }
52     }
53 };
54
55 int main() {
56     Book b1("A", "X", "111"), b2("B", "Y", "222");
57
58     Library lib(2);
59     lib.displayDetails();
60 }
```

Q3 Create a class *Account* with attributes *account number (of const long type)*, *transaction ID (of long type)*, *transaction type (credit/debit) (of string type)*, and *balance (of double type)*. The Account class should provide following methods:

- d. **long depositAmount(to, from, amount)**. The “to” and “from” are the account number of the sender and receiver. The function should return the transaction id.
- e. **long creditAmount(to, from, amount)** similar to the depositAmount function. This function should also return the transaction id.
- f. **void displayDetails()** should display all the details such as account number, remaining balance, transaction history (credited/debited). Make this function as the const function.
- g. In all the functions, make all the arguments as the const arguments.

Write a main function to demonstrate all the function calls. Create at least 5 accounts objects (using array or without array) to demonstrate the operations of credit and debit.

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  class Account {
5  private:
6      const long accNo;
7      long transactionID;
8      string type;
9      double balance;
10
11     static long tid;
12
13 public:
14     Account(long acc, double bal) : accNo(acc), balance(bal) {}
15
16     long depositAmount(const long to, const long from, const double amount)
17     {
18         balance += amount;
19         transactionID = ++tid;
20         type = "Credit";
21         return transactionID;
22     }
```

```
22
23   long creditAmount(const long to, const long from, const double amount) {
24       balance -= amount;
25       transactionID = ++tid;
26       type = "Debit";
27       return transactionID;
28   }
29
30   void displayDetails() const {
31       cout << "Account: " << accNo << endl;
32       cout << "Balance: " << balance << endl;
33   }
34 };
35
36 long Account::tid = 0;
37
38 int main() {
39     Account a1(101, 5000);
40
41     a1.depositAmount(101, 102, 1000);
42     a1.creditAmount(101, 102, 500);
43 }
```

input

Account: 101
Balance: 5500

...Program finished with exit code 0
Press ENTER to exit console.

Q4 Write a program to add data objects of two different classes using friend functions.

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  class B;
5
6  class A {
7      int x;
8  public:
9      A(int x) { this->x = x; }
10     friend int add(A, B);
11 };
12
13 class B {
14     int y;
15 public:
16     B(int y) { this->y = y; }
17     friend int add(A, B);
18 };
19
20 int add(A a, B b) {
21     return a.x + b.y;
22 }
23
24 int main() {
25     A a(5);
26     B b(10);
27
28     cout << "Sum = " << add(a, b);
29 }
```

input

Sum = 15

...Program finished with exit code 0
Press ENTER to exit console.

Q5 Define a class named *Complex* with properties (real and imaginary) and methods as per following details.

- Parameterized constructor* and *copy constructor* to initialize object values.
- void display ()* to display complex number.
- void sum (Complex, Complex)* to add two complex numbers (objects of

Complex class). Implement this function as the friend function.

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  class Complex {
5      int real, imag;
6
7  public:
8      Complex(int r, int i) {
9          real = r;
10         imag = i;
11     }
12
13     Complex(const Complex &c) {
14         real = c.real;
15         imag = c.imag;
16     }
17
18     void display() {
19         cout << real << " + " << imag << "i\n";
20     }
21
22     friend Complex sum(Complex, Complex);
23 };
24
25 Complex sum(Complex c1, Complex c2) {
26     return Complex(c1.real + c2.real, c1.imag + c2.imag);
27 }
28
29 int main() {
```

```
22     friend Complex sum(Complex, Complex);
23 };
24
25 Complex sum(Complex c1, Complex c2) {
26     return Complex(c1.real + c2.real, c1.imag + c2.imag);
27 }
28
29 int main() {
30     Complex c1(2,3), c2(4,5);
31
32     Complex c3 = sum(c1, c2);
33     c3.display();
34 }
```



A terminal window titled "input" with a standard macOS-style title bar (red, yellow, green buttons, zoom, and help icons). The terminal has a black background with white text. It displays the result of a calculation, followed by a green completion message and a prompt to press ENTER.

```
6 + 8i

...Program finished with exit code 0
Press ENTER to exit console.
```